Lab 6
Multithreading and Synchronization

Department of Computer Science
Information Technology University

September 21, 2025

Learning Objectives

Upon completion of this lab, students should be able to:

- Differentiate between processes and threads
- Create and manage threads using the pthreads library
- Identify and understand race conditions, deadlocks, and starvation
- Implement synchronization using mutexes and semaphores
- Solve classical synchronization problems
- Analyze thread behavior and synchronization effectiveness
- Use debugging tools like Valgrind and GDB to detect concurrency issues

1 Introduction to Threads and Processes

1.1 Process Concepts

A process is an instance of a program in execution. It represents a fundamental unit of work that the operating system schedules and manages. Each process has its own:

- Memory address space (code, data, stack, heap)
- System resources (open files, network connections)
- Security context (user ID, group ID, permissions)
- Execution state (register values, program counter)

Processes are isolated from each other by the operating system, which means one process cannot directly access another process's memory or resources without explicit inter-process communication (IPC) mechanisms.

1.2 Thread Concepts

Threads are lightweight units of execution within a process. Multiple threads can exist within a single process and share:

- The same memory address space
- Global variables and heap memory
- Open files and other system resources

However, each thread maintains its own:

- Stack for local variables
- Register values
- Program counter
- Scheduling properties (priority, policy)

The primary advantage of threads over processes is that thread creation, context switching, and communication are generally faster and more efficient due to shared resources.

2 POSIX Threads (pthreads) Programming

The POSIX threads library (pthreads) is a standardized API for thread management in UNIX-like systems. It provides functions for creating, managing, and synchronizing threads.

2.1 Basic Thread Management

2.1.1 Thread Creation

Threads are created using the `pthread_create()` function:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// Thread function prototype
void* thread_function(void* arg);

int main() {
    pthread_t thread_id;
    int argument = 42;

    // Create a new thread
    int result = pthread_create(&thread_id, NULL,
    thread_function, &argument);

    if (result != 0) {
        perror("Thread creation failed");
```

```
        exit(EXIT_FAILURE);
    }

    // Wait for the thread to complete
    pthread_join(thread_id, NULL);

    printf("Thread completed successfully\n");
    return 0;
}

// Thread function implementation
void* thread_function(void* arg) {
    int* value = (int*)arg;
    printf("Thread received value: %d\n", *value);

    // Optional: return a value
    return NULL;
}
```

Parameters of pthread_create():

- `thread`: Pointer to a `pthread_t` that will store the thread ID
- `attr`: Thread attributes (NULL for defaults)
- `start_routine`: Function pointer to the thread's entry point
- `arg`: Argument passed to the thread function

2.1.2 Thread Termination

Threads can terminate in several ways:

- Returning from the thread function
- Calling `pthread_exit()`
- Being canceled by another thread with `pthread_cancel()`

2.1.3 Thread Joining

The `pthread_join()` function allows one thread to wait for the completion of another thread:

```
void* thread_result;
pthread_join(thread_id, &thread_result);
```

This function blocks the calling thread until the specified thread terminates. If the thread function returns a value, it can be captured through the second parameter.

2.2 Thread Attributes

Thread attributes allow customization of thread behavior. Common attributes include:

- Detached state: Detached threads release resources automatically on termination
- Scheduling policy and priority
- Stack size
- Inheritance of attributes from creating thread

Example of setting thread attributes:

```
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
// Set other attributes as needed

pthread_t thread_id;
pthread_create(&thread_id, &attr, thread_function, NULL);

// Destroy attributes when no longer needed
pthread_attr_destroy(&attr);
```

3 Synchronization Primitives

Synchronization primitives ensure orderly access to shared resources in multithreaded programs.

3.1 Mutexes (Mutual Exclusion)

A mutex is a synchronization primitive that ensures only one thread can execute a critical section at a time. Threads attempting to lock an already locked mutex will block until the mutex becomes available.

3.1.1 Mutex Operations

```
#include <pthread.h>

// Declare and initialize a mutex
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// Alternative initialization with attributes
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutex_init(&mutex, &attr);

// Lock the mutex (block if already locked)
pthread_mutex_lock(&mutex);
```

```
// Try to lock without blocking (returns 0 if successful)
int result = pthread_mutex_trylock(&mutex);

// Unlock the mutex
pthread_mutex_unlock(&mutex);

// Destroy the mutex when no longer needed
pthread_mutex_destroy(&mutex);
```

3.2 Semaphores

A semaphore is a synchronization primitive that maintains a counter and a queue of waiting threads. Unlike mutexes, semaphores can allow multiple threads to access a resource simultaneously, up to a specified limit.

3.2.1 Semaphore Operations

```
#include <semaphore.h>

// Declare a semaphore
sem_t semaphore;

// Initialize semaphore
// pshared: 0 for threads, non-zero for processes
// value: initial semaphore value
sem_init(&semaphore, 0, 1);

// Wait (decrement) operation - blocks if value is 0
sem_wait(&semaphore);

// Try wait - doesn't block, returns -1 if value is 0
int result = sem_trywait(&semaphore);

// Post (increment) operation
sem_post(&semaphore);

// Get current semaphore value
int value;
sem_getvalue(&semaphore, &value);

// Destroy semaphore when no longer needed
sem_destroy(&semaphore);
```

4 Concurrency Problems

When multiple threads access shared resources without proper synchronization, several problems can occur.

4.1 Race Conditions

A race condition occurs when the behavior of a program depends on the relative timing of events, such as the order in which threads are scheduled. This typically happens when multiple threads access shared data without synchronization, and at least one thread modifies the data.

Listing 1: Example of a Race Condition

```c
#include <pthread.h>
#include <stdio.h>

int counter = 0;

void* increment_counter(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("Expected: 2000000, Actual: %d\n", counter);
    return 0;
}
```

Instructions:

1. Save the program as `race.c`.
2. Compile using the pthread library:

   ```
   gcc -o race race.c -lpthread
   ```

3. Run the program multiple times:

   ```
   ./race
   ```

4. Observe that the printed value of `counter` often differs from the expected `2000000`. Each execution may produce a different result.
5. Think about **why this happens**
6. Use `valgrind` with the thread error detection tool to analyze the race condition:

```
valgrind --tool=helgrind ./race
```

4.2 Deadlocks

A deadlock occurs when two or more threads are blocked forever, each waiting for a resource held by another. Four conditions must hold for a deadlock to occur:

1. Mutual exclusion: Resources cannot be shared
2. Hold and wait: Processes hold resources while waiting for others
3. No preemption: Resources cannot be forcibly taken
4. Circular wait: A circular chain of processes exists where each waits for a resource held by the next

Listing 2: Example of a Deadlock

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_function(void* arg) {
    pthread_mutex_lock(&mutex1);

    sleep(1);
    pthread_mutex_lock(&mutex2);


    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

void* thread2_function(void* arg) {
    pthread_mutex_lock(&mutex2);

    sleep(1);
    pthread_mutex_lock(&mutex1);
```

```
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Program completed\n");
    return 0;
}
```

4.3 Starvation

Starvation occurs when a thread is unable to gain access to a necessary resource and cannot make progress. This often happens when higher-priority threads continuously monopolize shared resources.

Listing 3: Example of Starvation

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* greedy_thread(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        printf("Greedy thread working...\n");
        sleep(1);
        pthread_mutex_unlock(&mutex);
        usleep(1000); ]
    }
    return NULL;
}

void* normal_thread(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        printf("Normal thread got access!\n");
        pthread_mutex_unlock(&mutex);
```

```
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, greedy_thread, NULL);
    pthread_create(&thread2, NULL, normal_thread, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    return 0;
}
```

5 Debugging Concurrent Programs

5.1 Using Valgrind to Detect Race Conditions

Valgrind is a powerful tool for detecting memory errors and race conditions in multithreaded programs.

```
# Install Valgrind
sudo apt-get install valgrind

# Compile with debug information
gcc -g -pthread program.c -o program

# Run with Helgrind (race condition detector)
valgrind --tool=helgrind ./program

# Run with DRD (thread error detector)
valgrind --tool=drd ./program
```

5.2 Using GDB for Debugging Deadlocks

GDB can be used to debug deadlocks by examining thread backtraces.

```
# Compile with debug information
gcc -g -pthread program.c -o program

# Run in GDB
gdb ./program

# In GDB, run the program
run
```

```
# When the program hangs, interrupt with Ctrl+C
# Get information about all threads
info threads

# Select a thread and get its backtrace
thread 1
bt

# Continue this for all threads to identify the deadlock
```

6 Lab Task

6.1 Task 1: Fixing Race Conditions

1. Compile and run the race condition example:

   ```
   gcc -g -pthread race_condition.c -o race_condition
   ./race_condition
   ```

2. Observe the inconsistent output

3. Use Valgrind to detect the race condition:

   ```
   valgrind --tool=helgrind ./race_condition
   ```

4. Fix the race condition using a mutex

5. Verify that the output is now consistent

6. Run Valgrind again to confirm the race condition is fixed

6.2 Task 2: Fixing Deadlocks

1. Compile and run the deadlock example:

   ```
   gcc -g -pthread deadlock.c -o deadlock
   ./deadlock
   ```

2. Observe how the program hangs

3. Use GDB to identify the deadlock:

   ```
   gdb ./deadlock
   # In GDB: run, then Ctrl+C, then info threads, then bt for each t
   ```

4. Identify the circular wait condition

5. Modify the code to prevent deadlock by enforcing a strict ordering of resource acquisition

6. Verify that the program no longer deadlocks

6.3 Task 3: The "Chaotic Bank" Simulation

Implement a simulation of a shared bank account with the following requirements:

- Initial balance: $10,000
- 1000 client threads, each performing 100 withdrawals of random amounts between $1 and $50
- 1 auditor thread that periodically checks the account balance to ensure the invariant: Final Balance == Initial Balance - Sum of All Withdrawals
- If the invariant is violated, the auditor should report an error and terminate the program

6.4 Task 4: Shared Common-room Problem

Implement a multithreaded simulation of a shared resource to demonstrate advanced synchronization techniques using mutexes and semaphores variables. The room has a limited capacity and enforces the rule that men and women cannot use it at the same time. This problem requires careful management of shared state and waiting queues to prevent race conditions, deadlocks, and starvation.

- Room capacity: 3 users at a time.
- Simulate 10 male threads and 10 female threads.
- Each thread attempts to use the room 5 times.
- Upon arriving, a thread must wait if the Room is occupied by the opposite gender or is full.
- When entering, the thread uses the room for a random time between 1 and 3 seconds (simulate with sleep).
- Use a mutex to protect shared variables tracking the current number of users, the current gender (or neutral if empty), and waiting counts for each gender.
- Use two semaphores: one for men waiting and one for women waiting.
- When the room becomes empty, prefer the gender with more waiting threads to prevent starvation (e.g., check waiting counts and broadcast to the preferred group).
- Log each action (arrive, enter, exit) with thread ID and gender to a console or file for debugging and verification.
- Ensure the program terminates correctly after all usages, and verify no mixed-gender usage occurs.

7 Bonus Challenge: Implementing Custom Synchronization Primitives

Implement your own mutex and semaphore using atomic operations in C. Replace the pthread mutex and semaphore implementations in the previous exercises with your custom implementations.